

Chapter 76B

Business Analyst Question Bank

Answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews · tell a BRD, FRD, and SRS apart without hesitating · write a user story with real acceptance criteria, not a vague wish

Approved chapter · v1

Contents

Chapter 76B. Business Analyst Question Bank

- 76.1 Requirements gathering
- 76.2 Basic-but-tricky BA questions
- 76.3 User stories and acceptance criteria
- 76.4 Process mapping
- 76.5 SDLC and where a BA fits
- 76.6 Gap analysis and UAT
- 76.7 Stakeholder management and pushback
- 76.8 Live-coding-style walk-throughs: full worked scenarios
- Common mistakes and how to spot them
- In the real world: the requirement that was actually three requirements
- Tools
- The project
- Final-week revision list
- Key terms
- Where this leads

Chapter 76B. Business Analyst Question Bank

Part VIII — The Interview Playbook

You will learn to: answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews · tell a BRD, FRD, and SRS apart without hesitating · write a user story with real acceptance criteria, not a vague wish · map a process the way a BA actually would, with decision points and swimlanes · handle "can you just change the number?" and other stakeholder pushback without folding or getting defensive.

How this chapter is built. Same format as Chapters 70–75: every core question leads with a "Remember it as..." hook, a one-line answer, a compact tier table. Rapid-fire sections are scan tables. This chapter has no code to execute; a requirements document, a process map, and a user story are judged on completeness and clarity, so every worked example here is a full, concrete artifact, not a description of one.

A note on sourcing. This chapter draws on two chapters elsewhere in this book: **Chapter 24 (Requirements, Storytelling & Stakeholders)**, already approved, and **Chapter 25 (The Business Analyst Track)**, not yet written at the time of this draft. Every "Learn it in" pointer below uses the topic and chapter number the blueprint already commits to (SDLC, process mapping, user stories, BRD/FRD/SRS, gap analysis, and UAT are all explicitly Chapter 25's stated scope), not a specific section number, since that text doesn't exist yet to check against. Flagged clearly for a re-check once Chapter 25 is drafted. Worked examples in this chapter use Riverstone's order-to-cash process, the same process Chapter 25's own planned project asks readers to map, so this bank stays consistent with that chapter even though it comes first.

76.1 Requirements gathering

Q76–001 · A stakeholder says "make the report better." How do you turn that into an actual requirement?

Remember it as: "Better" is a feeling, not a requirement. Your job is to find the specific, checkable thing hiding behind the vague word.

Answer in one line: Ask specifically what problem the current report causes them, day to day, and work backward from that concrete pain point to a testable requirement, rather than guessing at what "better" might mean.

Worked example, a real elicitation dialogue:

Analyst: "When you say 'better,' can you walk me through the last time the current report let you down?" **Stakeholder:** "Last Tuesday I needed last week's numbers by region, and I had to open it in Excel and build a pivot table myself, it took twenty minutes." **Analyst:** "So the requirement isn't 'better,' it's specifically: the report needs a region-level breakdown, viewable without any manual rework, available same-day."

Tier	What to say
Passes	Asks "what do you mean by better?" and accepts whatever vague answer comes back
Strong	Asks for a specific recent example of the pain, then restates it as a testable requirement in the stakeholder's own words
Extra points	+ [Business] the fix here isn't asking more questions in general, it's asking for a <i>specific instance</i> , since people are far better at describing one real moment than defining an abstract concept like "better" + [Validate] restating the requirement back to the stakeholder in one sentence and getting a "yes, exactly" confirms it was captured correctly before any work starts

Likely follow-ups: What if the stakeholder can't recall a specific example? How would you prioritize this requirement against three other vague asks from different stakeholders? **Red flag:** documenting "make the report better" as the requirement itself, or guessing at a specific fix with no elicitation at all. **Learn it in:** Chapter 24 (requirements gathering: turning asks into questions).

Q76-002 · What's the difference between a business requirement, a functional requirement, and a non-functional requirement?

Remember it as: *Business requirement: why we're doing this. Functional requirement: what the system must do. Non-functional requirement: how well it must do it.*

Answer in one line: A **business requirement** states the underlying business goal ("reduce time spent reconciling orders"); a **functional requirement** states what the system must specifically do to support that goal ("the system shall flag any order with a quantity mismatch between the PO and the invoice"); a **non-functional requirement** states a quality constraint on how it does that ("the flag must appear within 2 seconds of the invoice being uploaded").

Tier	What to say
Passes	Can define one of the three, conflates the other two
Strong	All three definitions above, with a matched example showing how one business requirement can produce several functional and non-functional requirements underneath it
Extra points	+ [Edge cases] a common real mistake: writing a functional requirement that's actually a design decision in disguise ("the system shall use a dropdown menu" describes a UI choice, not a requirement; "the system shall let a user select from a predefined list of statuses" is the actual requirement, implementation-neutral)

Likely follow-ups: Give a non-functional requirement that isn't about speed. Why does mixing requirement types in one document cause problems later? **Red flag:** writing an implementation detail (a specific button, a specific dropdown) as if it were the requirement itself. **Learn it in:** Chapter 25 (BRD, FRD, and SRS documents).

Rapid-fire, 76.1

#	Question	One-line answer	Extra point
Q76-003	What's a business rule, and how does it differ from a requirement?	A business rule is a constraint the business operates under regardless of any system ("orders over ₹5 lakh need manager approval"); a requirement describes what a specific system must do, often to <i>enforce</i> a business rule	[Business] business rules usually outlive any one system; requirements are tied to the specific project building or changing that system
Q76-004	What's elicitation, as a formal BA term?	The structured process of drawing out requirements from stakeholders, via interviews, workshops, document analysis, observation, or surveys, rather than requirements simply being handed over complete	[Business] different elicitation techniques suit different situations: a workshop surfaces conflicting stakeholder views fast; direct observation catches things nobody thinks to mention because it's "just how we do it"
Q76-005	Why interview more than one stakeholder for the same process?	Different roles see different parts of the process and often disagree about how it actually works, even for something they'd all claim to understand the same way	[Real evidence] a sales rep and a finance controller frequently describe the exact same order-approval process differently, and the gap between their descriptions is itself useful information
Q76-006	What's a requirements traceability matrix, in one sentence?	A table linking each requirement back to its business need and forward to the test case that verifies it, so nothing gets built without a reason or shipped without being checked	[Business] it's the tool that answers "why are we building this" and "how do we know it worked" for every single requirement, at once

76.2 Basic-but-tricky BA questions

Q76-007 · What's the difference between a BRD, an FRD, and an SRS?

Remember it as: BRD is for the business, FRD is for the build, SRS is for the engineers, each one more technical and more detailed than the last.

Answer in one line: A **BRD** (Business Requirements Document) captures the business need and goals, written for business stakeholders, largely non-technical; an **FRD** (Functional Requirements Document) translates those goals into specific functional behaviors the system must support, still readable by a business audience but more detailed; an **SRS** (Software Requirements Specification) is the most technical of the three, detailed enough for engineers to actually build from, often including data structures, interfaces, and technical constraints.

Tier	What to say
Passes	Knows all three acronyms exist, can't distinguish their actual audience or level of detail
Strong	The audience-and-detail-level distinction above, with a one-line example of the same requirement expressed at each level
Extra points	+ [Edge cases] not every organization uses all three; some combine BRD and FRD into one document, or skip a formal SRS entirely and go straight to user stories (§76.4) for agile teams: naming that this varies by organization shows real experience, not just memorized definitions

Likely follow-ups: Which of the three would a project sponsor sign off on? Which would a developer actually build from? **Red flag:** treating the three as interchangeable synonyms. **Learn it in:** Chapter 25 (BRD, FRD, and SRS documents).

Q76-008 · Is a user story the same thing as a requirement?

Remember it as: A user story is a conversation starter with acceptance criteria attached. A requirement can be a much heavier, more formal document. They're related, not identical.

Answer in one line: Not exactly: a **user story** is a short, deliberately incomplete description of a need from a user's perspective ("As a [role], I want [goal], so that [benefit]"), meant to prompt a conversation and get refined collaboratively with the team; a **requirement** in the traditional sense is meant to be complete and unambiguous on its own, without needing further conversation to be understood.

Tier	What to say
Passes	Treats "user story" and "requirement" as interchangeable words for the same thing
Strong	The completeness distinction above: a user story is intentionally a placeholder for a conversation, not a finished specification
Extra points	+ [Business] this is exactly why a user story needs acceptance criteria (Q76-013) attached before it's actually "done": the criteria are what turn the conversation starter into something testable, closing the gap between the two

Likely follow-ups: In what kind of project would you use formal requirements documents instead of user stories, or both together? **Red flag:** treating a bare user story ("As a user, I want a better dashboard, so that I can see my data") as if it were already a complete, actionable requirement. **Learn it in:** Chapter 25 (use cases and user stories).

Rapid-fire, 76.2

#	Question	One-line answer	Extra point
Q76-009	Use case vs. user story?	A use case describes a full interaction between an actor and a system, including alternate and exception flows, in more formal, structured detail / a user story is a short, single-sentence expression of a need	[Business] use cases suit complex, multi-step system interactions; user stories suit iterative, conversation-driven agile work
Q76-010	What's an assumption, in a requirements document, and why document it explicitly?	Something taken as true without confirmation, stated openly so it can be challenged or validated before it silently becomes a wrong foundation for the whole project	[Learn it in] Chapter 69's Move 2 (state assumptions), the identical discipline applied to formal documentation instead of a live answer
Q76-011	What's the difference between "in scope" and "out of scope," and why does an explicit out-of-scope list matter?	In scope is what the project will deliver; out of scope is what it explicitly will not, stated up front to prevent scope creep and manage expectations before, not after, stakeholders assume otherwise	[Business] an unstated out-of-scope item is the single most common source of a stakeholder feeling blindsided at delivery

#	Question	One-line answer	Extra point
Q76-012	What's a stakeholder register?	A structured list of everyone with an interest in the project, their role, their influence, and how and when they need to be communicated with	[Business] distinguishing high-influence, high-interest stakeholders (manage closely) from low-influence, low-interest ones (monitor only) is the core of practical stakeholder prioritization

76.3 User stories and acceptance criteria

Q76-013 · Write a complete user story with acceptance criteria for: "sales reps want to know which of their accounts are at risk of churning"

Remember it as: A user story with no acceptance criteria is a wish. Acceptance criteria are what turn a wish into something a developer can build and a tester can check off.

Answer in one line: The story states the role, the goal, and the benefit in one sentence; the acceptance criteria list the specific, testable conditions that must be true for the story to be considered done.

Worked example:

User story: As a sales rep, I want to see a list of my accounts flagged as at-risk of churning, so that I can prioritize retention calls before an account is lost.

Acceptance criteria:

- Given I am logged in as a sales rep, when I open my accounts dashboard, then I see a visible "At Risk" flag on any account with a churn-risk score above the defined threshold.
- Given an account is flagged as at-risk, when I click on it, then I see the top 2-3 factors driving that risk score.
- Given the churn-risk score changes, when it crosses the threshold in either direction, then the flag updates within 24 hours, not instantly, matching the model's daily refresh cycle.
- Given I have no at-risk accounts, when I open my dashboard, then I see a clear "no accounts currently at risk" message, not an empty or broken-looking screen.

Tier	What to say
Passes	Writes the one-sentence story, no acceptance criteria at all
Strong	The story plus at least two or three "Given/When/Then" criteria covering the main happy path
Extra points	+ [Edge cases] the fourth criterion above (the empty state) is exactly the kind of edge case a strong BA adds unprompted: what happens when there's <i>nothing</i> to show is easy to forget and easy for a developer to leave broken if it's never specified + [Business] the third criterion ties the acceptance criteria to a real technical constraint (a daily refresh cycle) rather than an unrealistic implicit expectation of real-time updates

Likely follow-ups: How would you prioritize this story against three others in a sprint? What would make you split this into two smaller stories instead of one? **Red flag:** acceptance criteria so vague they can't actually be tested ("the dashboard should work well"). **Learn it in:** Chapter 25 (use cases and user stories, acceptance criteria).

Rapid-fire, 76.3

#	Question	One-line answer	Extra point
Q76-014	What does "Given/When/Then" mean in acceptance criteria?	Given (the starting context) / When (the action taken) / Then (the expected result), a structured format (Gherkin syntax) for writing testable criteria	[Business] this structure is what lets a QA engineer write a test case almost directly from the acceptance criteria with no further translation
Q76-015	What's the INVEST mnemonic for a good user story?	Independent, Negotiable, Valuable, Estimable, Small, Testable	[Edge cases] a story that fails "Independent" (it can't be built without another story finishing first) often signals it should be resequenced or the dependency made explicit
Q76-016	Why should a user story avoid specifying a particular UI element?	The story should describe the need, not the implementation, so the team retains freedom to choose the best actual design; specifying a "dropdown" locks in a decision that isn't the BA's or the story's to make	[Learn it in] Q76-002's implementation-vs-requirement distinction, the same discipline
Q76-017	What's a spike, in agile terminology?	A time-boxed research or investigation task, done specifically because a story can't be sized or written properly until an open technical question is answered first	[Business] a spike is the honest alternative to guessing at a story's size or acceptance criteria when genuine uncertainty exists

76.4 Process mapping

Q76-018 · Map Riverstone's order-to-cash process at a high level, and explain what a swimlane adds that a simple flowchart doesn't

Remember it as: A flowchart shows what happens. A swimlane shows what happens AND whose job it is, which is usually the more useful question in a real business process.

Answer in one line: A basic flowchart shows the sequence of steps; a **swimlane diagram** adds horizontal or vertical lanes, one per role or department, so each step visibly sits inside the lane of whoever's responsible for it, immediately surfacing handoffs between departments, which is usually exactly where a real process breaks down.

Worked example, order-to-cash, swimlane by role:

Sales rep lane: receives customer order → enters order in CRM **Warehouse lane:** receives order → picks and packs → ships, updates status **Finance lane:** generates invoice on shipment → sends to customer → tracks payment → reconciles on receipt **Customer lane:** places order → receives goods → pays invoice

Tier	What to say
Passes	Draws a linear flowchart of the process with no indication of who's responsible for each step
Strong	The swimlane version above, correctly separating steps by which team owns them
Extra points	+ [Business] the handoff <i>between</i> lanes (sales → warehouse, warehouse → finance) is almost always where delays and errors concentrate in a real process, an insight a plain flowchart hides entirely by not distinguishing ownership at all + [Edge cases] a real map would also show what happens on an exception path (a rejected order, a payment dispute), not just the clean happy path shown above

Likely follow-ups: Where in this process would you look first for a delay-reduction opportunity, and why? What would a BPMN version of this add beyond a plain swimlane diagram? **Red flag:** a process map showing only the happy path, with no exception handling at all. **Learn it in:** Chapter 25 (process mapping: flowcharts, swimlanes, BPMN basics).

Rapid-fire, 76.4

#	Question	One-line answer	Extra point
Q76-019	What's a decision point (or gateway) in a process map?	A point where the process branches based on a condition (approved vs. rejected, in-stock vs. backorder), shown as a diamond in most notations	[Edge cases] every decision point needs every branch mapped, including the "unhappy" ones; a map that only shows the approved path is incomplete
Q76-020	What does BPMN stand for, and what does it add over a generic flowchart?	Business Process Model and Notation, a standardized symbol set (specific shapes for events, gateways, tasks, and messages between participants) that makes a process map unambiguous across different tools and readers	[Business] standardization matters most when a process map needs to be handed to an external vendor or IT team who wasn't in the room when it was drawn
Q76-021	What's the difference between an "as-is" and a "to-be" process map?	As-is documents the process exactly as it currently, actually works, warts and all; to-be documents the proposed, improved version	[Edge cases] skipping the as-is map and jumping straight to to-be is a common mistake: you can't credibly propose an improvement to a process you never actually documented as it really runs
Q76-022	Why might the as-is process, once actually mapped, surprise the people who work in it every day?	People executing a process day to day often only know their own step; mapping the whole thing end to end frequently reveals workarounds, redundant approvals, or handoffs nobody had the full picture of before	[Real evidence] this is one of the most consistently reported "aha" moments in real BA work: the map itself is often the deliverable that creates the most value, before any redesign happens at all

76.5 SDLC and where a BA fits

Q76-023 · Walk through the phases of the SDLC, and say specifically what a BA does in each

Remember it as: A BA's role doesn't stop after requirements are written. It runs the whole length of the SDLC, in a different form at every stage.

Answer in one line: Across the typical SDLC phases (planning, requirements/analysis, design, development, testing, deployment, maintenance), a BA leads or heavily contributes to planning and requirements, stays involved during design to confirm the solution still matches the requirement, supports development by answering clarifying questions as they come up, leads UAT (§76.6) during testing, and often supports the go-live and early maintenance period by triaging whether an issue is a genuine defect or a new requirement in disguise.

Tier	What to say
Passes	Names the SDLC phases correctly, describes the BA's role as "writing requirements" with no involvement in the other phases
Strong	The phase-by-phase involvement above, correctly identifying that a BA's role changes shape but doesn't disappear after requirements are handed off
Extra points	+ [Business] the triage role at the maintenance stage (is this a bug, or a new requirement someone's calling a bug) is a genuinely common, underappreciated part of the job, and naming it specifically signals real experience rather than a textbook answer

Likely follow-ups: How does a BA's role differ in a Waterfall project versus an Agile one? What happens when a developer's question during the build phase reveals the original requirement was actually ambiguous? **Red flag:** describing the BA role as ending once requirements are signed off. **Learn it in:** Chapter 25 (the software development life cycle).

Rapid-fire, 76.5

#	Question	One-line answer	Extra point
Q76-024	Waterfall vs. Agile, from a BA's perspective?	Waterfall: requirements are finalized once, up front, before development starts / Agile: requirements evolve iteratively across sprints, with the BA continuously refining the backlog	[Business] most real organizations run a hybrid, not a pure version of either, worth naming rather than presenting the two as a strict binary
Q76-025	What's a sprint backlog, and who owns prioritizing it?	The set of user stories committed to the current sprint; typically the Product Owner owns final prioritization, with the BA heavily informing it through requirements and stakeholder input	[Edge cases] a BA and a Product Owner role can overlap significantly or be entirely separate people depending on the organization
Q76-026	What's a Definition of Done, and why does a team need to agree on one explicitly?	A team-wide, explicit checklist of what "finished" means for any story (code reviewed, tested, documentation updated, and so on), agreed once so it doesn't get silently redefined story by story	[Business] without an explicit Definition of Done, "done" quietly means something different to the developer, the BA, and the stakeholder, and that gap surfaces late, expensively

#	Question	One-line answer	Extra point
Q76-027	What's a change request, and why does even a small one need a formal process?	A formally logged request to modify already-approved scope, requirements, or a signed-off deliverable; a formal process (however lightweight) protects against silent scope creep and keeps a record of who approved what	[Business] the process doesn't need to be heavy for a small change, but skipping it entirely for "just this once" is exactly how scope creep starts

76.6 Gap analysis and UAT

Q76-028 · What's a gap analysis, and walk through one for Riverstone's order-to-cash process

Remember it as: Gap analysis is just "where are we now" minus "where do we need to be," made explicit and specific enough to act on.

Answer in one line: A **gap analysis** compares the current (as-is) state against the desired (to-be) state, identifying the specific differences that need to be closed, each one becoming a candidate requirement or improvement initiative.

Worked example, Riverstone's order-to-cash process:

As-is: Invoices are generated manually by finance after a shipment notification email from the warehouse, typically 1–2 days after actual shipment. **To-be:** Invoices generate automatically the moment the warehouse marks an order as shipped in the system. **Gap:** No current system integration between the warehouse's shipment status and finance's invoicing system; manual, email-triggered handoff is the bottleneck. **Resulting requirement:** The system shall automatically trigger invoice generation within 1 hour of an order's status changing to "Shipped."

Tier	What to say
Passes	Defines gap analysis correctly in the abstract, can't apply it to a concrete example
Strong	The full worked example above: as-is, to-be, the specific gap, and the requirement it produces
Extra points	+ [Business] the gap analysis doesn't stop at identifying the difference, it names the <i>root cause</i> (no system integration, not "people are slow"), which is what actually makes the resulting requirement solve the real problem rather than a symptom of it

Likely follow-ups: How would you prioritize this gap against several others found in the same analysis? What would you check before assuming automatic invoicing is actually the right fix? **Red flag:** a gap analysis that identifies a difference but never traces it to an actionable, specific requirement. **Learn it in:** Chapter 25 (gap analysis).

Q76-029 · What's UAT, who does it, and why can't developers or QA alone replace it?

Remember it as: UAT isn't "does the system work." It's "does the system actually do what the business needed," and only the business can really answer that.

Answer in one line: **User Acceptance Testing** is the final validation stage, done by real business users (not developers or QA), checking whether the delivered system actually meets the original business need, not just whether it's technically functioning correctly, since a system can pass every technical test while still failing to solve the real problem it was built for.

Tier	What to say
Passes	"UAT is when users test the system before launch" (correct, misses <i>why</i> it can't be skipped or delegated to QA)
Strong	The distinction above: QA verifies the system works as specified; UAT verifies the specification itself was actually right, which only the people with the original business need can judge
Extra points	+ [Business] a system can pass 100% of its QA test cases and still fail UAT, because the test cases were built from a requirement that, it turns out, didn't fully capture what the business actually needed, exactly the kind of gap Q76-001's elicitation discipline is meant to prevent upstream

Likely follow-ups: What would you do if UAT reveals the original requirements were wrong, this late in the project? How would you structure a UAT test plan? **Red flag:** treating UAT as a redundant, box-checking repeat of QA testing. **Learn it in:** Chapter 25 (UAT).

Rapid-fire, 76.6

#	Question	One-line answer	Extra point
Q76-030	What's a UAT test case, and how does it differ from a QA test case?	Written from the business user's real workflow and language, checking business outcomes / QA test cases are written from the technical specification, checking system behavior	[Learn it in] Q76-013's acceptance criteria are effectively pre-written UAT test cases, when written well
Q76-031	What happens if UAT fails close to a planned go-live date?	The defect gets triaged: is it a genuine bug (fix and retest), a missed requirement (assess impact and possibly delay), or a misunderstanding of scope (clarify and possibly defer to a later phase)	[Business] the temptation to launch anyway and "fix it after" needs to be weighed honestly against the real cost of the specific gap found, not treated as automatically acceptable or automatically a blocker
Q76-032	Who signs off on UAT completion?	Typically the business stakeholder or sponsor who owns the requirement, not IT or the project manager, since sign-off is specifically confirming the business need was met	[Business] this sign-off is the formal moment accountability for "does this actually work for the business" transfers, worth taking seriously rather than as a formality

76.7 Stakeholder management and pushback

Q76-033 · A stakeholder says "can you just change the number to make it look better?" How do you respond, live?

Remember it as: Don't refuse and don't comply silently. Ask what's actually driving the request, then respond to that, not to the literal words.

Answer in one line: Ask what's behind the request (are they worried the number looks bad to their own boss, do they suspect the number itself is wrong, is there missing context that would explain it) before either agreeing or refusing, since the literal request is almost never actually what they need.

Worked example, talked through live:

"I'd say: 'I want to understand what's driving that, can you tell me more? If the number's wrong, I'll absolutely fix it, and I'd want to find the root cause. If it's right but looks concerning, I can add context that helps explain it rather than changing the number itself, since changing a correct number isn't something I can do.' Most of the time what they actually want is help explaining a real number, not a fabricated one, and naming that directly usually resolves it without an awkward confrontation."

Tier	What to say
Passes	Either flatly refuses ("I can't do that") with no further conversation, or quietly changes the number to avoid conflict
Strong	The diagnostic question above, distinguishing "the number might be wrong" from "the number is right but needs context" before responding either way
Extra points	+ [Business] offering a legitimate alternative (adding context, a footnote, a caveat) gives the stakeholder something real they can use, rather than leaving them with nothing after a flat refusal, which is often what actually causes the request to escalate or repeat

Likely follow-ups: What if the stakeholder insists after you've explained you can't change a correct number? How would you document this exchange, if at all? **Red flag:** silently complying with a request to alter accurate data, or a defensive, unhelpful flat refusal with no attempt to solve the real underlying concern. **Learn it in:** Chapter 24 (handling pushback and "can you just change the number?").

Rapid-fire, 76.7

#	Question	One-line answer	Extra point
Q76-034	Two stakeholders give you contradicting requirements. What do you do?	Don't silently pick one; bring both stakeholders together (or relay both positions clearly) so the conflict is resolved explicitly, with a documented decision, not quietly absorbed by the BA	[Business] an undocumented, silently-resolved conflict tends to resurface later when the stakeholder who "lost" notices the outcome

#	Question	One-line answer	Extra point
Q76-035	A powerful stakeholder wants a feature that conflicts with the actual data or user needs. How do you push back?	Lead with evidence, not opinion: show the specific data or user feedback that conflicts with the request, and frame it as a shared problem to solve together, not a personal disagreement	[Learn it in] Chapter 24's storytelling discipline (bottom line up front, one message) applied to a difficult conversation instead of a report
Q76-036	What's the risk of only communicating with the most senior stakeholder on a project?	Requirements end up shaped by whoever has the most authority to speak up, not necessarily whoever has the most accurate day-to-day knowledge of the actual process	[Learn it in] Q76-005's multiple-stakeholder-interview discipline
Q76-037	How would you communicate a delay to a stakeholder who's expecting on-time delivery?	As early as possible, with the specific reason, the revised timeline, and what's being done about it, not a vague "it's taking longer than expected" with no further detail	[Business] early, specific bad news is almost always received better than late, vague bad news

76.8 Live-coding-style walk-throughs: full worked scenarios

Q76-038 · Full scenario, talked through live: elicit requirements for automating Riverstone's manual invoice-matching process

What they're really testing: whether a full elicitation-to-requirement cycle can be run start to finish, live, not just described as a concept.

Talked through live, start to finish:

"First, I'd ask finance to walk me through exactly how they currently match invoices to purchase orders, step by step, including what they do when something doesn't match, since the exception path usually matters more than the happy path. Say they tell me: they manually compare invoice line items against the PO in two separate systems, and when quantities or prices don't match exactly, they flag it for a supervisor. I'd ask how often that happens, since if mismatches are rare, automation has a smaller payoff than if they're common. Say it's about 15% of invoices. I'd then ask what 'match' actually means precisely, exact figures, or within some tolerance, since 'close enough' matching is a very different, harder requirement than exact matching. From there I'd draft a first-pass requirement: 'The system shall automatically compare each invoice's line items against its corresponding PO, flag any mismatch beyond a defined tolerance for supervisor review, and auto-approve matches within tolerance,' and take that back to finance to confirm before writing it up formally."

Extra-points moves demonstrated: **[Clarify]** asked about the exception path, not just the happy path, unprompted. **[Business]** asked for the mismatch frequency specifically because it changes the ROI case for automating this at all. **[Validate]** ended by proposing to confirm the draft requirement with the stakeholder before treating it as final, rather than writing it up unilaterally.

Likely follow-ups: How would you write the acceptance criteria for this requirement? What would change about your approach if this were a brand-new process instead of automating an existing manual one? **Red flag:** drafting a requirement after asking only about the happy path, with no question about what happens when something doesn't match. **Learn it in:** §76.1 (elicitation) and §76.6 (gap analysis), pulled together into one live scenario.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Documenting a vague ask as the requirement itself	Nobody can tell what "done" actually means	Ask for a specific example of the pain, restate as a testable requirement
Treating BRD/FRD/SRS or use case/user story as interchangeable	Wrong document for the audience, confusion downstream	Know the audience and detail level each one is for
A user story with no acceptance criteria	The story can't actually be tested or marked done	Always attach Given/When/Then criteria, including edge cases
A process map showing only the happy path	Exception handling gets built as an afterthought, or not at all	Map every branch at every decision point, including rejections
Assuming the BA's job ends once requirements are signed off	The delivered system quietly drifts from the original need	Stay involved through design, build, and UAT
Skipping the as-is map and jumping straight to to-be	A proposed improvement that doesn't actually address the real current process	Always document as-is first, however tempting it is to skip
Silently complying with (or flatly refusing) "can you just change the number"	Either damaged data integrity or an unresolved, escalating stakeholder relationship	Diagnose what's actually driving the request before responding

In the real world: the requirement that was actually three requirements

Farah, now working as a Business Analyst after the career transition Chapters 8, 9, and 68 followed, is asked to gather requirements for "a way to see which customers haven't ordered recently." She resists the urge to write that sentence down as the requirement and instead asks three different stakeholders, separately, what they'd actually do with that information.

Sales wants it to prioritize outreach calls, and only cares about customers likely to still be recoverable. Finance wants it to flag accounts for a different reason entirely: identifying dormant accounts that should be written off for accounting purposes, with no interest in whether they're "recoverable." Customer success wants it for a third reason: proactively checking in with accounts before they become a support escalation, regardless of order recency alone.

What looked like one requirement was genuinely three, each needing a different definition of "haven't ordered recently," a different output format, and a different owner. Farah's actual deliverable ends up being three separate, precisely scoped requirements instead of one vague one, and the project's initial one-week estimate for "the recency report" becomes a more honest three-week estimate for three real features. Her manager's note: *"The original ask would have shipped something nobody could actually use well. Three real requirements, properly scoped, took longer to define but were faster to build right the first time."*

Tools

No software specific to this chapter. A whiteboard or a simple diagramming tool (Lucidchart, draw.io, or even PowerPoint) for process maps; a shared document for BRDs, FRDs, and user stories; Chapter 70's Excel/Sheets skills for a requirements traceability matrix in practice.

The project

Goal: run this chapter's full method on a real process, start to finish.

1. Pick a real, recurring process you're familiar with (at work or elsewhere), and interview at least two people involved in different steps of it.
 2. Map it as-is, including at least one exception path, using a swimlane format.
 3. Write one user story with full Given/When/Then acceptance criteria for a specific improvement to that process.
 4. Draft the requirement, in one sentence, that a gap analysis of the same process would produce.
 5. Write the "can you just change the number" response you'd actually give, for a real or hypothetical version of that request in your own context.
-

Final-week revision list

Q76-001, Q76-002, Q76-007, Q76-008, Q76-013, Q76-018, Q76-023, Q76-028, Q76-029, Q76-033, Q76-038.

Key terms

elicitation · business requirement · functional requirement · non-functional requirement · business rule · requirements traceability matrix · BRD · FRD · SRS · user story · use case · INVEST · Given/When/Then · Definition of Done · spike · swimlane diagram · BPMN · decision point (gateway) ·

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against, especially Move 1 (clarify) throughout §76.1 and §76.8.
- **Chapter 75, Product Sense, Metrics, Case Studies & Guesstimates**, already covers case-structure and stakeholder reasoning from the diagnostic-case angle; this chapter covers the same discipline from the documentation and process side.
- **Chapter 24 and Chapter 25** teach every technique this bank draws on, in full; this chapter tests it, and (for Chapter 25 specifically) was written ahead of that chapter's own draft, so its worked examples deliberately match Chapter 25's own planned Riverstone order-to-cash project.
- **Chapter 78, Automation & Integration Question Bank**, picks up directly from this chapter's automation-requirement questions (Q76-038) at the technical implementation level.